ACIT 4850 – Lab 1 – Source Code Management

Instructor	Mike Mulder (mmulder10@bcit.ca)
Total Marks	10
Due Dates	Demo and Submission Due by End of Lesson 2 Class: • Jan. 13th for Sets B and C (Tuesday) • Jan. 14th for Set A (Wednesday)

Applicable Requirements

- **REQ1010** - The Enterprise Development Environment shall be prototyped on Microsoft Azure cloud infrastructure.
- **REQ1020** – The Enterprise Development Environment shall have a Source Code Management capability. The Source Code Management tool will be GitLab. Note: The GitLab Community Edition (free) will be sufficient for the prototype environment.
- **REQ1030** - The Source Code Management tool shall be integrated with the single sign-on services provided by the Microsoft Azure cloud infrastructure. Note: The oauth integration from Azure Active Directory (AD) will be used for the prototype.
- **REQ1040** – The Source Code Management tool shall allow software development teams to create source code repositories and logically group them. Note: Source code repositories are Projects in GitLab and groupings of repositories are Groups in GitLab. Logical groupings may be by software product and common infrastructure.
- **SEC1010 –** All password credentials will be stored in a password safe.
- **SEC1020** – All web applications and API endpoints shall be encrypted (i.e., https endpoints). Note: Self-signed certificates are sufficient for the prototype environment.

Group Formation

For this lab and many subsequent labs you will work with a partner in order to:

- Share computing resources (and any associated costs).
- Work together for more complex tasks.
- Divide up the work for larger tasks.

Find a partner and signup with that partner in a Lab Group for your set on the Learning Hub.

For most labs, you will work with your partner and can choose to share Azure computing resources. So you can decide to only use the Azure account for one group member for each lab, or balance resources across both accounts.

It is **<u>your responsibility</u>** to manage the resources on Azure as you will be billed for your usage above the free tier and any credits you are provided by Microsoft. *Make sure you disable any resources, such as virtual machines when not being used.*

Cloud Provider Setup

Signup for the Microsoft Azure for Student Free Account Credit. No credit card is required. This gives you the basic Azure free services plus a $100 account credit (if you have not already signed up for this offer).

https://azure.microsoft.com/en-us/free/students/

You will then be using for Azure for Students subscription to cover most of your Azure costs for this course. One you have exceeded the credits, you will have to provide a credit card and pay for any non-free services yourself.

Note: Do not use your BCIT e-mail for your Azure user as this can cause problems since it is associated with BCIT's active directory. Create an Azure account using another e-mail. If you do use an Azure account associated with your BCIT e-mail, you will have to create a new Tenant in Azure Active Directory and apply your Student Subscription under that tenant.

If you decide to work on the same Azure account with your partner, one person will need to sign-up for Azure and add the other as a co-administrator. To add a co-administrator, login to your Azure dashboard.

Add your partner to Azure Active Directory (AD):

- All services -> Azure Active Directory
- Users
- New Guest User
- Add your partner's information and invite them to your Azure AD
- Your partner should then get an e-mail inviting them to Azure.

Make your partner a Contributer (can see your resources like VMs):

- All services -> Subscriptions -> Azure for Students
- Access control (IAM)
- Add -> Add Role Assignment -> Privileged administrator roles (tab) -> Click Contributor. Once selected click "Members"
- Select the user you just added to Azure AD to make them a Contributor member.

The Azure for Student credit balance can be monitored here:
https://www.microsoftazuresponsorships.com/

Familiarize Yourself with Azure

Microsoft Azure has most of the equivalent services to AWS that you may be familiar with. The following website provide the equivalent Azure service for most common AWS services:

https://docs.microsoft.com/en-us/azure/architecture/aws-professional/services

The two main Azure services you will use for this lab are:

- Azure Active Directory – for single sign-on

- Virtual Machines – to host a GitLab instance

Source Code Management Setup

Step 1 - VM Creation

Note: GitLab will be quite slow to install and run on the free tier virtual machine as it likely does not have enough RAM. Recommend using a more capable VM (such as B2ms with 8GB of RAM) and stopping the instance when not actively being used.

Select the Virtual Machines service in Azure and Add a new Virtual Machine.

Create a Linux Virtual Machine (using the Virtual Machines Service) with the following specifications under your Azure for Students subscription:

Basics:

- Subscription: Azure for Students
- Resource group (Create new): GitLabCE
- Virtual machine name: GitLabCE
- Region: (US) West US (or whichever has the cheapest monthly cost for your VM size)
- Image: Ubuntu 24.04 LTS – x64 Gen2
- Size: Standard_B2ms
- Authentication Type: SSH public key (create an SSH key if necessary and paste in the public key or have it create one for you)
- Inbound Ports: Allow HTTP (80), HTTPS (443), SSH (22)

Networking:

- Make sure Inbound Ports 80, 443 and 22 are open

Leave everything else at the defaults and Create the Virtual Machine. This may take a few minutes.

Step 2 - Set the VM's DNS Name

When the VM is created, view the newly created VM's resources. It should already be assigned a Public IP address. However, this address may change when the VM is stopped and restarted.

Select the *Not Configured* link next to the DNS name label under the VM's details. Set the DNS name label to the following:

gitlab-acit4850-groupX (where X is your Group number)

Your full DNS name will be something like:

gitlab-acit4850-group1.westus2.cloudapp.azure.com

Make sure you record it as you will need it for the GitLab installation.

Select Save and then close the Configure view (i.e., select X in the top right corner)

Step 3 - GitLab CE Installation

Using your ssh key, ssh into your VM.

Follow the instructions at the following URL to install GitLab CE your Ubuntu 24.04 LTS (or later) VM. Read the notes below before running through the instructions.

https://docs.gitlab.com/install/package/ubuntu/?tab=Community+Edition

IMPORTANT – Make sure to install a specific version (17.1.0) of GitLab for this lab as Azure Active Directory used below is deprecated. We will upgrade in a future lab.

```
sudo EXTERNAL_URL="https://<Your VM DNS>" apt install gitlab-ce=17.1.0-ce.0
```

Notes:

- **Make free Community Edition tab is selected when you run the download and installation commands.**
- Use the DNS name you recorded above for your GitLab URL (i.e., https://gitlab-acit4850-group1.westus2.cloudapp.azure.com) as your EXTERNAL_URL. Make sure to include the leading https://
- A LetsEncrypt SSL certificate should automatically provisioned for your GitLab CE installation. If you get a validation error, verify the Networking configuration of your VM. Ports 80 and 443 should be open for inbound access.

Once the installation is done, go to the specified file on your VM to find the root password (/etc/gitlab/initial_root_password). Make sure you record the root password somewhere safe so you can find it again (i.e., a password safe).

You can now go to https://<Your_VM_DNS> and login to the GitLab web application with root as the username and your root password.

Step 4 – Single Sign-on

Because our Enterprise Development Environment will eventually be used by many product team members and will contain multiple applications, we do not want them to have to manage usernames and passwords for each application. Therefore, we want to integrate with a single sign-on capability.

For this prototype, we will use Azure Active Directory to manage users and to enable single sign-on through Oauth2.

In the Azure Portal:

- Go to Microsoft Entra ID (use the search bar to find it as necessary). You should be in your Default Directory (this won't work in the BCIT Directory since it's locked down).
- Select App registrations in the left side menu (under Manage)
- Select New Registration
    - Name: GitLabCE
    - Supported Account Types: Accounts in this organizational directory only (the default)
    - Redirect URI Web - https://<Your VM DNS>/users/auth/azure_activedirectory_v2/callback

- Click Register. The details of your new registration should show up.

Display name	: GitLabCE	Supported account types	: My organization only
Application (client) ID	: eee805ae-6ec3-462d-a88a-159a3594871f	Redirect URIs	: 1 web, 0 spa, 0 public client
Directory (tenant) ID	: 28f302e9-5ae6-4011-9484-cce45b537891	Application ID URI	: Add an Application ID URI
Object ID	: 06c712d2-4bc8-4f06-9d26-a2bce5c4b3ae	Managed application in l...	: GitLabCE

- Select Certificates & secrets on the left side menu
- Add a New client secret
  - Set the Description to client_secret and set Expires to 6 months
  - Note the Value of the client_secret can only be copied when first created. If you don't get the value at that time, you'll need to delete it and recreate it.

+ New client secret

Description	Expires	Value	ID
client_secret	12/31/2299	.1~**********	373e6316-ff83-4c44-9de5-145286fa4ed0

- You need the following from the details of your new registration:
  - Application (client) ID
  - Directory (tenant) ID
  - client_secret

The above will not work if your Azure account is associated with your BCIT e-mail.

SSH into your GitLabCE VM:

- Open /etc/gitlab/gitlab.rb as root (i.e., use sudo). We're going to make the file look something like this:

Note: Replace **azure_oauth2** with **azure_activedirectory_v2** below as azure_oauth2 has recently been deprecated.

```
# OAuth Config
gitlab_rails['omniauth_enabled'] = true
gitlab_rails['omniauth_allow_single_sign_on'] = ['azure_oauth2']
gitlab_rails['omniauth_sync_email_from_provider'] = 'azure_oauth2'
gitlab_rails['omniauth_sync_profile_from_provider'] = ['azure_oauth2']
gitlab_rails['omniauth_sync_profile_attributes'] = ['name','email']
gitlab_rails['omniauth_block_auto_created_users'] = false
gitlab_rails['omniauth_auto_link_ldap_user'] = true
gitlab_rails['omniauth_external_providers'] = ['azure_oauth2']
gitlab_rails['omniauth_providers'] = [
  {
    "name" => "azure_oauth2",
    "args" => {
      "client_id" => "$CLIENT_ID",
      "client_secret" => "$CLIENT_SECRET",
      "tenant_id" => "$TENANT_ID",
    }
  }
]
```

Note: There are underscores above in azure_oauth2.

There is an existing section in the file with the above settings. You can uncomment it and update the values to match those above.

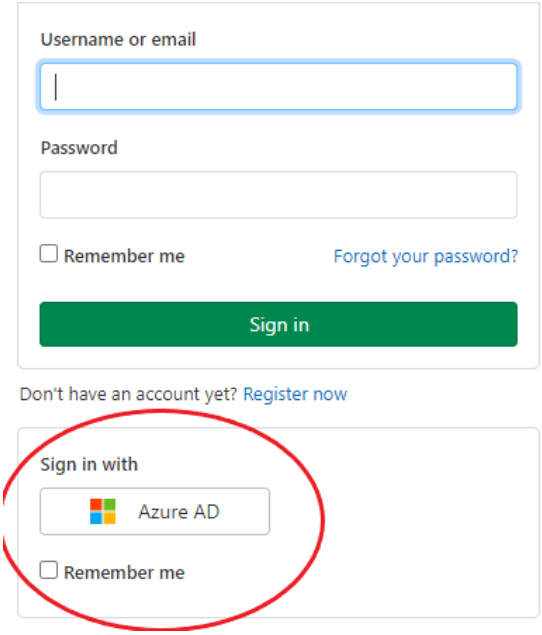
The $CLIENT_ID, $CLIENT_SECRET and $TENANT_ID should be replaced with your values from your App Registration.

- Reconfigure GitLab by running the following as root (i.e., use sudo):

```
gitlab-ctl reconfigure
```

This command will report any errors with your configuration that you need to fix.

When you go to your GitLab application, it should now allow you to login with Oauth.



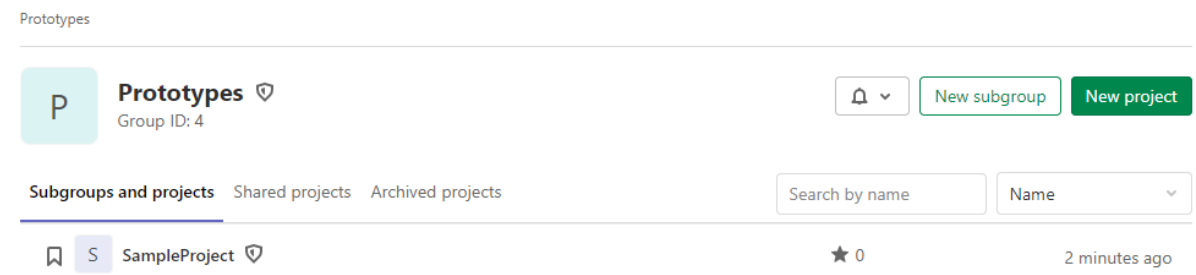
Make sure both you and your partner can login to GitLab using Oauth. If not, then you need to debug your configuration.

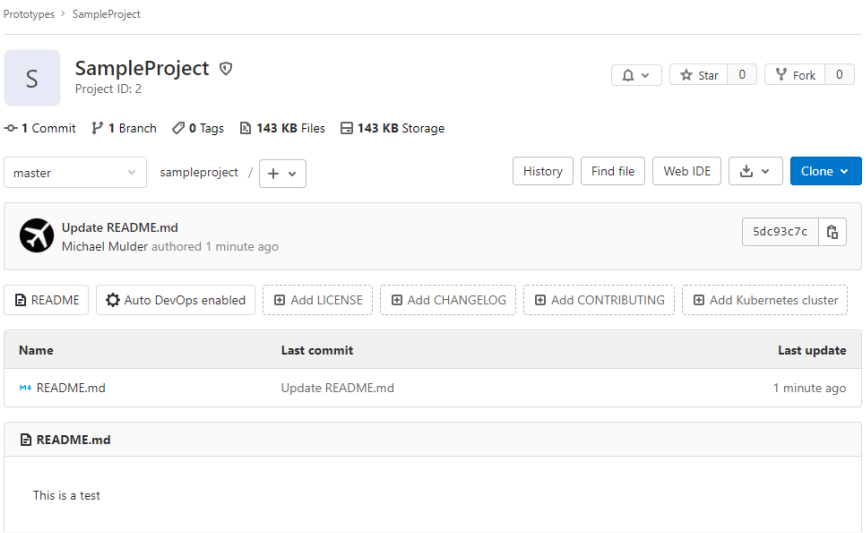
Also make sure to disable self sign-up in GitLab as we only want access to GitLab from our Azure users.

Step 5 – GitLab Groups and Projects

Under your user account (not root), login to GitLab and create the following:

- A Group called Prototypes (Visibility should be Internal)
    - Make sure both users, if applicable, are added to the group
- A Project called SampleProject (Visibility should be Internal) in the Prototypes group
- A Readme file in SampleProject





Note: When you login in for the first time with your user account using Oauth, an account is created for your user in GitLab. You must login as root after this account is created and make your user an admin in order to be able to create Groups and Projects.

Verify that you can clone the SampleProject repository locally to both yours and your partner's laptops.

Note: To be able to clone the repo locally you need to setup credentials in GitLab for your user. You can either upload your ssh public key, create an access token or (I believe) there is a way to use your Oauth credentials directly.

You are now done and ready to demo next class. **Make sure you shutdown any Azure resources (i.e., the VM) to conserve your credits and free tier usage.**

You can setup your VM to automatically shut down at the certain time of day (i.e., midnight) if it's been left running.

Go to your VM in the Azure Portal. Make sure it's stopped.

On the left side, selected Auto Shutdown under Operations and enable it.

Demo, Grading and Submission

A demo of your lab against the applicable requirements which will determine your grade on the lab. All mandatory requirements must be met otherwise you will receive zero on the lab. You can re-demo the lab if you haven't met the mandatory requirements, up to the last class before the midterm week, but you will lose 20% every week late.

Reqt.	Mandatory	Demo	Marks
REQ1010	Yes	• Show your Azure dashboard (both students). • Show your running VM.	2
REQ1020	Yes	• Show your running GitLab instance at the DNS name of your server from Azure	2
REQ1030	Yes	• Show that you can login to GitLab with your Azure user using Oauth (both students).	2
REQ1040	Yes	• Show your Group and Project in GitLab and your local clone of the Project (both students).	2
SEC1010	Yes	• Show that your GitLab endpoint is encrypted (https).	1
SEC1020	No	• Show that you have used ssh keys for ssh access to your VM. • Describe how you are securely storing your GitLab root password	1
Total			**10**

<u>Submit a screenshot of your GitLab Group/Project to the Lab1 dropbox under Activities -> Assignments on D2L.</u>